Assignment 3 — Technical Documentation

Project: Museum of Science Boston Educator Portal — Search & AI backend
Course: LIS 467, Simmons University, Fall 2026
Company: Agentic Curriculum Designers, Inc.
Assignment due: Day 1 of Week 10 (20% of grade)

Architecture overview

```
assignment3/
├── webserver/
│   ├── data/
│   │   ├── documents.json        ← 206 searchable documents (source of truth)
│   │   ├── index.json            ← vector index (built from documents.json, gitignored)
│   │   └── sources/              ← raw source files and intermediate data
│   ├── scripts/
│   │   ├── build_documents.py    ← generates documents.json from source files
│   │   ├── buildIndex.js         ← generates index.json from documents.json
│   │   ├── fetch_extracts.py     ← downloads text from linked PDFs/pages
│   │   ├── fetch_site_pages.py   ← fetches official descriptions from mos.org/yes.mos.org
│   │   ├── build_nav.py          ← builds the site navigation from the IA spreadsheet
│   │   └── (source files used at build time only)
│   ├── src/
│   │   └── embeddings.js         ← shared embedding logic (Node build; browser uses CDN)
│   ├── public/
│   │   ├── ask.php               ← RAG API endpoint (runs on Simmons PHP)
│   │   ├── index.html            ← Dashboard / home page
│   │   ├── search-results.html   ← Educator search results page
│   │   ├── css/                  ← colors.css (mood board), layout.css (structure)
│   │   ├── images/               ← logo.png and exhibits/ photos
│   │   └── js/                   ← empty, reserved for search scripts
│   ├── simmons-test/
│   │   ├── test.html             ← browser-side vector search test harness
│   │   ├── test.js               ← test logic (ES module)
│   │   └── data/index.json       ← copy of index for standalone test
│   ├── package.json
│   └── .gitignore
└── LINUX_STEPS.md                ← steps for pushing Linux-built pages
```

The search works in two modes:

1. **Browser-only vector search** — the browser loads `index.json`, downloads the MiniLM model from a CDN, and ranks results by cosine similarity. No server needed beyond static file hosting.

2. **PHP RAG endpoint** — `ask.php` does keyword retrieval from `index.json` and sends the top results to Claude Haiku for a grounded natural-language answer.

Both modes are confirmed working on `web.simmons.edu`.

`data/documents.json`

Generated by: `scripts/build_documents.py`
Size: 206 documents, ~198 KB

This is the single content database for all search. Each document is a JSON object:

```
{
  "id": "yes-016",
  "type": "yes-unit",
  "title": "Engineering Earthquake-Resistant Buildings",
  "category": "YES - YES Middle School (Grades 6–8)",
  "text": "Engineering Earthquake-Resistant Buildings is a YES Middle School unit ...",
  "url": "https://yes.mos.org/curricula/yes-middle-school/engineering-earthquake-resistant-
        buildings"
}
```

Field	Purpose
`id`	Unique identifier, prefixed by type (`yes-`, `exh-`, `ste-`, `doc-`)
`type`	`yes-unit`, `yes-overview`, `exhibit`, `ste-standard`, `ste-overview`, `research-document`
`title`	Display name shown in search results
`category`	Sub-grouping shown in results (e.g. "YES - YES Middle School (Grades 6–8)")
`text`	The text that gets embedded and searched. Combines title, description, aligned standards and related exhibits
`url`	The real page on `yes.mos.org`, `mos.org`, `doe.mass.edu`, or the source PDF

Document counts by type:

Type	Count	Source
`yes-unit`	53	YES unit pages on `yes.mos.org`, including career explorations and CS modules
`yes-overview`	7	One per YES section (preschool, elementary, middle, enrichment, etc.)
`exhibit`	26	MOS exhibit pages from `mos.org/visit/exhibits/`
`ste-standard`	20	Massachusetts STE standards from the alignment table
`ste-overview`	1	Overall MA STE framework objectives
`research-document`	99	Curated STEM PDFs from NASA, USGS, FEMA, NPS, EPA, Code.org, CS Unplugged, etc.

Why `text` is composite: a vector search works by semantic similarity to the embedded `text` field. By composing text that includes the unit description, its aligned MA STE standard, and related exhibits, a single query like "light

and optics" can surface a YES unit, its matching standard, and a museum exhibit in one search, with no separate queries.

`data/index.json`

Generated by: `scripts/buildIndex.js` (Node.js)
Size: ~2.4 MB
Gitignored — rebuild on each machine with `npm run build-index`

Each entry in `index.json` is the same object as `documents.json` plus an `embedding` array:

```
{
  "id": "yes-016",
  "title": "Engineering Earthquake-Resistant Buildings",
  ...
  "embedding": [0.032, -0.041, 0.018, ...]   ← 384 numbers, L2-normalised
}
```

The 384-number vector is produced by the `Xenova/all-MiniLM-L6-v2` model, a compact sentence-transformer that maps any text to a fixed-size point in 384-dimensional space. Documents with similar meaning end up close together in that space. At search time, the query is mapped to the same space and the closest documents (by cosine similarity = dot product of unit vectors) are returned.

`scripts/build_documents.py`

Language: Python 3
Dependencies: `openpyxl`, `requests`, `beautifulsoup4`
Run: `python3 scripts/build_documents.py`

Reads four source markdown files and the IA spreadsheet and writes `data/documents.json`. It does **not** need network access (all network fetching is done by the two `fetch_` scripts).

Key sections

`parse_yes()` — reads `Youth Engineering Solutions (YES) Curricula List.md` section by section. Each `##` heading becomes a YES section (Preschool, Elementary, etc.); bullet points and table rows become unit names and grade bands.

`table_rows(path)` — parses a markdown table into a list of row arrays, stripping markdown link syntax (`[text](url)` → `text`) so the content is clean plain text.

`xlsx_categories()` — opens `Information_Architecture_MOS_Ed_portal.xlsx` with openpyxl and reads the hyperlink targets. The spreadsheet's cell hyperlinks are the definitive list of 100 curated research document URLs. Each URL is mapped to its Level 4 category (e.g. "Computer Science, AI, Robotics").

`norm(name)` — normalises a unit name to a lookup key by lowercasing, removing parenthetical content, and dropping common stop words like "engineering" and "yes". Used to match a unit name in the YES list against entries in the exhibit and standards alignment tables.

`matches(unit_norm, names)` — checks whether a normalised unit name appears in a list of normalised activity names extracted from the exhibit or standards table. Used to cross-link: "this exhibit matched this YES unit" and "this YES unit aligns with this standard".

Main loop — YES units: For each unit, `text` is built from:

1. The unit name and section (e.g. "Engineering Earthquake-Resistant Buildings is a YES Middle School unit from the Museum of Science, Boston.")
2. The official description fetched from `yes.mos.org` (from `site_pages.json`)
3. The grade band if available
4. Any matching MA STE standard with its description
5. Any related museum exhibit names

Main loop — exhibits: For each exhibit row from the alignment table, `text` is built from the alignment rationale, the official MOS description (from `site_pages.json`), whether it is a curriculum gap, and the primary MA STE standard code.

Main loop — research documents: Uses `link_replacements.json` to substitute working URLs for the 78 dead links found in the original spreadsheet, and `pdf_extracts.json` to include the opening text of each document so the search can match against actual content.

`scripts/fetch_extracts.py`

Language: Python 3
Dependencies: `openpyxl`, `requests`, `beautifulsoup4`, `pdftotext` (system)
Run: `python3 scripts/fetch_extracts.py`
Output: `data/sources/pdf_extracts.json`

Downloads each linked document from the IA spreadsheet (using working replacements from `link_replacements.json` where the original is dead) and extracts the opening text:

- For PDFs: pipes the file through `pdftotext -l 3` (first 3 pages only) for speed.
- For HTML pages: strips navigation and script tags with BeautifulSoup and extracts body text.
- Uses two User-Agent strings in sequence (browser UA, then `curl/8.7.1`) because some government servers block one or the other.
- Saves up to 1,500 characters of text per document. `build_documents.py` includes the first 700 of those in the `text` field.

`scripts/fetch_site_pages.py`

Language: Python 3
Dependencies: `requests`, `beautifulsoup4`

Run: `python3 scripts/fetch_site_pages.py`
Output: `data/sources/site_pages.json`

Fetches every page listed in `data/sources/site_links.json` — the real `yes.mos.org` unit pages, `mos.org` exhibit pages, and the DESE standards page. For each page it saves:

- `title` — from the `og:title` meta tag or `<title>` element
- `description` — from the `name="description"` meta tag (the official one-line summary)
- `body` — up to 1,500 characters of main-content text (with nav/footer/script stripped)

`build_documents.py` uses `description` to populate each YES unit's and exhibit's `text` field with the museum's own words.

`scripts/buildIndex.js`

Language: Node.js (ES module)
Dependencies: `@xenova/transformers`
Run: `npm run build-index`
Source: copied from `467-vectorsearch-demo/scripts/buildIndex.js`

```
// For each document in documents.json:
const embedding = await embedText(`${doc.title}. ${doc.text}`);
// embedText() returns a 384-element Float32 array, L2-normalised
indexed.push({ ...doc, embedding });
```

The document title is prepended to the text before embedding. This is a standard trick: it gives the title extra weight in the semantic representation, so a query mentioning the exact unit name gets a strong hit even if it uses different vocabulary in the body.

The model (`Xenova/all-MiniLM-L6-v2`) is a 23 MB ONNX-quantised version of the `all-MiniLM-L6-v2` sentence transformer. It produces identical vectors whether run in Node.js or in the browser via the same `@xenova/transformers` library, confirmed by the test harness (similarity 1.0000).

`src/embeddings.js`

Language: Node.js / browser ES module (same file works in both)
Source: copied from `467-vectorsearch-demo/src/embeddings.js`

```
export async function embedText(text) {
  const embedder = await getEmbedder();           // lazy-loads the model once
  const output = await embedder(text, {
    pooling: "mean",     // average token vectors → single document vector
    normalize: true      // L2-normalise → dot product = cosine similarity
  });
  return Array.from(output.data);
}

export function cosineSimilarity(a, b) {
  let dot = 0;
  for (let i = 0; i < a.length; i++) dot += a[i] * b[i];
  return dot;            // both vectors are already unit-length, so dot = cosine
}
```

Mean pooling averages the per-token vectors into one vector representing the whole sentence. L2 normalisation scales the vector to unit length, which turns cosine similarity (normally `dot(a,b) / (|a| × |b|)`) into a plain dot product — cheaper to compute, especially in PHP and at 206 × 384 multiplications.

`simmons-test/test.html` and `test.js`

Purpose: confirm the browser-side vector search stack works on a given host before building the real pages.

The test runs eight checks in sequence:

#	Check	What it proves
1	Page served over http(s)	Not a `file://` open; a real web server
2	ES modules and WebAssembly	Browser can run the transformer model
3	Load `data/index.json`	Server serves JSON correctly; 206 docs, 384 dims
4	Load Transformers.js from CDN	`cdn.jsdelivr.net` reachable; library works
5	Download and start the model	Model downloads and initialises in the browser
6	Embed a query	384-number unit vector produced correctly
7	Browser vectors match Node index	Vectors are identical (similarity 1.0000)
8	Semantic search returns relevant results	"earthquake" query returns earthquake unit first

All eight passed on `web.simmons.edu` over HTTP. The key finding: WebAssembly (required by Transformers.js) works on plain HTTP on this server, and the model cached in the browser between runs.

`public/ask.php`

Language: PHP 8.4
Deployed to: `public_html/ask.php` on `web.simmons.edu`
API key: `/users/c/colek/.anthropic_key` (permissions 604; never in git or `public_html`)

This is the RAG (Retrieval-Augmented Generation) endpoint. It:

1. Receives a POST request with `{"query": "...", "persona": "educators"|"students"|"parents"}`.
2. Loads `index.json` from `public_html/data/index.json`.
3. Filters documents by persona.
4. Scores every matching document by keyword overlap with the query.
5. Passes the top 6 to Claude Haiku with a grounded system prompt.
6. Returns `{answer, persona, results[]}` as JSON.

Section by section

Persona filtering:

```php
const PERSONA_TYPES = [
    'students'  => ['exhibit', 'yes-unit', 'yes-overview'],
    'parents'   => ['exhibit'],
    'educators' => null,   // null = all types (206 docs)
];
```

Educators see everything. Students see exhibits and YES units. Parents see exhibits only. This matches the three radio buttons on the wireframe's "Ask LLM" panel.

Keyword retrieval:

```php
$terms = array_filter(preg_split('/\W+/u', strtolower($query)));
foreach ($docs as $doc) {
    $hay  = strtolower($doc['title'] . ' ' . $doc['category'] . ' ' . $doc['text']);
    $hits = 0;
    foreach ($terms as $t) {
        if (strlen($t) > 2 && str_contains($hay, $t)) $hits++;
    }
    $scored[] = ['doc' => $doc, 'score' => $hits / count($terms)];
}
```

PHP can't run the MiniLM model (which requires ONNX/WebAssembly), so it falls back to term frequency. The score is the fraction of query words (longer than 2 characters) that appear in the document. A future improvement would have the browser do vector ranking and send the top IDs to PHP, which then only needs to call the LLM.

System prompt:

```
You are a helpful assistant for the Museum of Science, Boston educator portal.
Answer in 2–4 sentences, citing only the provided resources by their title.
End with a one-sentence suggestion for how to explore further on the portal.
Do not invent resources not listed below.
```

The "do not invent" instruction is the grounding constraint — it prevents the model from hallucinating resources that don't exist in the index.

Claude Haiku call:

```
$payload = json_encode([
    'model'      => 'claude-haiku-4-5-20251001',
    'max_tokens' => 350,
    'system'     => $system,
    'messages'   => [['role' => 'user', 'content' => $user_msg]],
]);
```

350 tokens is enough for 2-4 tight sentences. Claude Haiku is chosen for speed (sub-second) and cost (~$0.0003 per call at this token count).

Graceful degradation: if the API call fails (network error, rate limit, etc.), `ask.php` still returns the keyword search results with a note in the `answer` field. The page is never broken by an LLM failure.

`data/sources/link_replacements.json`

78 of the 100 research document URLs in the original IA spreadsheet returned 404 or 403. Six parallel background agents searched for working replacements, each handling 13 documents. Every proposed URL was independently verified with `curl` (HTTP 200, correct content type, topic confirmed). The replacements are stored here so `build_documents.py` can use them without re-fetching.

Notable findings:

- Most NASA PDFs at the old `nasa.gov/wp-content/uploads` paths have moved. New paths are at `assets.science.nasa.gov`.
- All FEMA PDFs now return 403 with a browser User-Agent but 200 with `curl/8.7.1`.
- Code.org lesson plans are served as `application/octet-stream` but are valid PDFs.
- CS Unplugged PDFs are available but at different paths than the originals.
- 6 items have no replacement PDF and are represented by HTML resource pages instead.
- 1 document (USGS Earthquakes Student Information Pamphlet) had no working replacement and was removed from the index.

`data/sources/site_links.json`

Maps every YES unit name, exhibit name and curriculum section to its real URL. Built by inspecting the unit directory page at `yes.mos.org/curricula/unit-directory` and the MOS sitemap at `mos.org/sitemap.xml`.

Notable mappings:

- The four Wee Engineer preschool units (Loud Noisemakers, Wind-Powered Fans, Wrecking Balls, Rafts) all share one page: `yes.mos.org/curricula/preschool/wee-engineer`.

- Theater of Electricity links to the Lightning! show page (`mos.org/visit/live-presentations/lightning`), not an exhibit page.
- Math Moves!: Ratio & Proportion shares the Mathematica page.
- Nanotechnology, A Bird's World, Foucault Pendulum and Soundstair & Kinetic Art have no dedicated pages on the current MOS site; they link to the exhibits listing.

`scripts/build_nav.py`

Input: `assignment3/Information_Architecture_MOS_Ed_portal.xlsx` **Output:** the navigation block inside `public/index.html` and `public/search-results.html` **Requires:** `pip install openpyxl` (Python 3.10+, otherwise standard library only)

The spreadsheet is the single source of truth for the site navigation. The script reads it, builds a tree, renders the tree as nested `<ul>` menus, and writes the result into both pages. Nothing is hand-edited in the pages' nav.

```
cd assignment3/webserver/scripts
python3 build_nav.py                                  # update both pages (depth 4)
python3 build_nav.py --depth 3                        # persona > main nav > dropdown only
python3 build_nav.py --dry-run                        # print the HTML, change nothing
python3 build_nav.py --xlsx IA.xlsx --pages ../public/index.html
```

Flag	Default	Meaning
`--xlsx`	`assignment3/Information_Architecture_MOS_Ed_portal.xlsx`	Spreadsheet to read
`--pages`	`public/index.html` and `public/search-results.html`	Pages to update (one or more)
`--depth`	`4`	Levels to show: 2 = persona and main nav, 3 = plus dropdown, 4 = plus category or link, 5 = everything
`--dry-run`	off	Print the nav HTML instead of writing files

The spreadsheet it expects

One sheet. A header row whose first cell starts with `Level 1`, then one row per nav entry:

Level 1 (Persona Nav)	Level 2 (Main Nav)	Level 3 (dropdown)	Level 4 (Category/Direct Link)	Level 5 (Direct Link)	URL Path	Page Purpose
Educators	Youth Engineering Solutions	Preschool	Wind-Powered Fans		/yes/preschool/wind-fans	

Columns A–E hold the path through the menu (a row stops at its first empty level) and column F holds the URL. Rows above the header (Global Utilities and Dashboard) are skipped, since those are the search box, chat and logo

link rather than menu items. Rows whose Level 1 is not one of the four personas are ignored.

Code walkthrough

Constants.

- `PERSONA_ORDER` is `["Students", "Parents", "Educators", "About"]`, the left-to-right order in the wireframe. The sheet lists Educators first, so the script ignores sheet order for the top bar.
- `PERSONA_LABELS` turns the sheet's bare names into the wireframe labels "For Students", "For Parents" and "For Educators". "About" is left alone.
- `LOGO_HTML` is the logo link (`images/logo.png`, back to `index.html`), the first item in the bar.
- `START` and `END` are the `<!-- NAV:START -->` and `<!-- NAV:END -->` markers.

`Node` is one menu entry: a `label`, a `children` dict keyed by label, and a list of `urls`.

- `child(label)` returns the existing child with that label or creates it. This is how rows that share a prefix (every row starting `Educators > Youth Engineering Solutions > Preschool`) merge into one branch.
- `href` decides the link target. If the node has exactly one distinct URL and no children, it links there. Otherwise (a category that holds other items, or a node with no URL) it returns `#`.

`clean(value)` turns a cell into a stripped string, or `None` if it is empty, so blanks and whitespace cells behave the same.

`read_tree(xlsx_path)` opens the workbook with openpyxl and finds the header row (first cell starts with `Level 1`; exits with a message if there is none). For each row below it:

1. Take columns A–E as the path, and column F as the URL.
2. Skip the row if Level 1 is not a known persona.
3. Walk from the root, calling `child()` for each non-empty level and stopping at the first empty one.
4. Append the URL to the node where the walk ended.

It returns the root `Node`.

`make_href(url)` leaves `http(s)` and `#` links alone and strips the leading `/` from internal paths, so `/exhibits/#` becomes `exhibits/#`. This keeps links relative so they resolve under `~colek/` on the Simmons server.

`render_item(node, depth, max_depth)` renders one `<li>`. External links get `target="_blank" rel="noopener"`. If the node has children and `depth < max_depth`, the `<li>` gets class `has-sub` and a nested list from `render_list`. Labels and URLs are passed through `html.escape`.

`render_list(nodes, depth, max_depth)` renders a `<ul class="sub">`. If every item in it is a leaf (such as the 26 exhibits), it also gets `menu-scroll` so the CSS can cap its height and scroll. Lists that contain flyouts must not scroll, because `overflow` would clip the flyouts.

`render_nav(root, max_depth)` builds the whole bar. For each persona in `PERSONA_ORDER` it:

- collapses a same-named leaf child (`About > About`) into the persona link itself, so "About" links straight to `/about`;
- applies `PERSONA_LABELS` to the visible text;
- adds a dropdown from `render_list` when the persona has children and `max_depth > 1`.

It returns the logo plus the persona list wrapped in `<nav class="site-nav">`, with the markers included.

`inject(page, nav_html)` reads a page, replaces everything from `START` to `END` with the new markup (a regex with `re.S`, using a lambda replacement so backslashes in the HTML are not treated as escapes), and writes it back. A page without both markers makes the script exit with an error rather than guess where the nav goes.

`main()` parses the flags, builds the tree, renders it, and either prints it (`--dry-run`) or injects it into each page. Running it twice produces identical files.

Generated markup

```
<!-- NAV:START -->
<nav class="site-nav" aria-label="Main">
<a class="logo" href="index.html"><img src="images/logo.png" alt="ACI Inc. logo – back to
        dashboard"></a>
<ul class="nav-list">
<li class="has-sub"><a href="#">For Educators</a><ul class="sub">
<li class="has-sub"><a href="#">Exhibits</a><ul class="sub menu-scroll">
<li><a href="exhibits/#">Gaia Globe</a></li>
...
</ul></li>
...
</ul></li>
</ul>
</nav>
<!-- NAV:END -->
```

The classes are styled in `public/css/layout.css`: `.site-nav` is the bar, `.nav-list` the top-level items, `.sub` the dropdown panels (hidden until hover or keyboard focus), `.has-sub` marks items with a ▸ flyout, and `.menu-scroll` limits long lists to 70% of the viewport height.

Known limits

- Many nav targets are still placeholders (`#` or `exhibits/#`) because those pages do not exist yet.
- Level 5 is hidden by default: it is mostly 100 PDF links under Resources, which would make an unusable menu.
- If the sheet gains a fifth persona, add it to `PERSONA_ORDER` (and `PERSONA_LABELS` if it needs a "For ..." label), or its rows are ignored.
- After changing the spreadsheet, re-run the script and redeploy both HTML files.

Front-end pages: `public/index.html` and `public/search-results.html`

Both pages implement the wireframes in item 7 of the A2 deliverable (Dashboard / Home Page and Educator Search Result Page). They are plain static HTML with no JavaScript yet; the search form is wired to `search-results.html` by a normal GET request, and the dynamic content is still placeholder text (see "Not yet wired" below).

Both pages load the same two stylesheets, in this order:

```
<link rel="stylesheet" href="css/colors.css">
<link rel="stylesheet" href="css/layout.css">
```

The two stylesheets

File	Role
`public/css/colors.css`	The mood board. Defines the nine palette colors and two fonts as CSS custom properties on `:root`, utility classes (`.bg-*`, `.text-*`), and the global base styles for `body`, headings and links.
`public/css/layout.css`	Page structure. Every component on the two pages is styled here, and it reads the palette through `var(--...)` rather than hard-coded hex values. It must load second because it depends on the variables.

Palette variables (from the A2 mood board):

Variable	Hex	Variable	Hex	Variable	Hex
`--blue-dark`	`#00205C`	`--gray-dark`	`#575757`	`--purple-dark`	`#57007F`
`--blue-mid`	`#00318E`	`--gray-mid`	`#898989`	`--purple-mid`	`#7900B2`
`--blue-bright`	`#0042BC`	`--gray-light`	`#BCBCBC`	`--purple-bright`	`#9C00E5`

Font variables: `--font-heading` is Cascadia Code (monospace fallback) and `--font-body` is Verdana (Geneva / sans-serif fallback).

Global rules in `colors.css` that apply to every page without any class: `body` uses `--font-body` in `--gray-dark`; `h1`–`h6` use `--font-heading` in `--blue-dark`; links are `--blue-bright` and turn `--purple-bright` on hover. This is why headings and paragraph text look correct even where `layout.css` sets no color.

The `.bg-*` and `.text-*` utility classes in `colors.css` are available for later pages but are not currently used by either page.

`layout.css` also sets `box-sizing: border-box` and `margin: 0` on `body` for the whole site.

Elements shared by both pages

Element	Markup	Styling and notes
Navigation bar	`<header>` containing `nav.site-nav`	**Generated by `scripts/build_nav.py` from the IA spreadsheet** (see the `build_nav.py` section above); do not edit between the `<!-- NAV:START -->` and `<!-- NAV:END -->` markers. `layout.css` `.site-nav`: flex row, `--gray-dark` background, white bold links. Its dropdown rules (`.nav-list`, `.sub`, `.has-sub`, `.menu-scroll`) show submenus on hover or keyboard focus, using `--gray-dark` panels with a `--purple-mid` accent and `--purple-dark` item highlight. At 800 px or narrower the bar wraps.
Logo link	`a.logo > img` (`images/logo.png`)	`.site-nav .logo img` sets the height to 48 px. It links to `index.html`; on the results page this is the wireframe's "Link to Dashboard". The logo was extracted from the A1 PDF.

Element	Markup	Styling and notes
Nav links	Students, Parents, Educators, About (in wireframe order)	The persona dropdowns, with items and flyouts taken from the IA spreadsheet. Internal URLs are placeholders (`exhibits/#`, `#`) until those pages exist.
Ask LLM panel	`section.ask-llm` containing a `form`	`.ask-llm`: `--purple-dark` background, white centered text, white `h2`. `fieldset` becomes a centered flex row of radio buttons; the search input is up to 24 rem wide. The form uses `action="search-results.html" method="get"`, so submitting sends `audience` and `q` in the URL. The legend is `hidden` and exists for screen readers.
Footer	`footer.site-footer`	`.site-footer`: flex row, `--gray-dark` background, white text. Left side: FAQ heading, three placeholder social icons (`.social`, white squares and a circle), and legal/copyright text. Right side: `.map` with a grey `.map-box` placeholder for a real map.
Buttons	`.btn`, `.btn-dark`	`.btn`: white pill with a `--gray-dark` border. `.btn-dark` swaps to a `--blue-dark` fill with white text.
Image placeholder	`.placeholder-img`	Grey (`--gray-light`) box with centered `--gray-mid` text, used wherever a photo is not available.
Photo credit	`small.credit`	`.credit`: small `--gray-mid` text, shown as "Photo: Museum of Science" under each mos.org photo.

`index.html` (Dashboard / Home Page)

`<main>` has four sections, top to bottom:

1. **Hero** (`section.hero`). `.hero` gives a `--gray-light` background with centered text. The `h1` "Museum of Science" is sized in `.hero h1` and picks up Cascadia Code and `--blue-dark` from `colors.css`. Below it is a "Featured Exhibit" link styled `.btn.btn-dark`.
2. **Ask LLM** (`section.ask-llm`). Described above. Radio options are Students, Parents and Educators, with none selected, as drawn in the wireframe.
3. **Upcoming Exhibits** (`section.exhibits`).
   - `.exhibits` centers the heading and adds padding.
   - `div.carousel` is a horizontal flex row with `overflow-x: auto` and `scroll-snap-type: x mandatory`, so cards scroll sideways and snap into place. It uses native scrolling; there are no arrow buttons yet.
   - It contains 26 `figure.card` elements, one per `exhibit` in `documents.json`, in the same order. Each card is `flex: 0 0 min(300px, 70%)` wide with `scroll-snap-align: start`.
   - 18 cards have a photo from `images/exhibits/`: an `img` (16:9, `object-fit: cover`), a `figcaption` with the exhibit name (`.card figcaption`), and a `small.credit` line.
   - The other 8 exhibits (Hall of Human Life, Theater of Electricity, Math Moves!, Nanotechnology, A Bird's World, Foucault Pendulum, Soundstair & Kinetic Art, AlegreMENTE) have no usable photo on mos.org. They show a `.placeholder-img` labelled with the exhibit name and have no credit line.
   - `div.dots` is a row of six small circles (`.dots span`) that mimic the wireframe's pagination indicator. The first is `--purple-mid`, the rest `--gray-light`. They are decorative (`aria-hidden`) and do not yet track the scroll position.
4. **Events** (`section.events`). `.events` is a centered, wrapping flex row.
   - `.calendar` is a 280 px column. Its `h3` "Calendar Widget" is a `--purple-dark` bar. `.calendar-grid` is a 6-column CSS grid of 30 `--gray-light` squares (`span`), a placeholder for a real calendar.
   - `.event-info` holds the "Event Info" `h2`, placeholder text, and a "Learn More" `.btn`.

`search-results.html` (Educator Search Result Page)

`<main class="results-layout">` is a two-column CSS grid (`3fr 2fr`) that collapses to one column at 800 px or narrower.

Left column (`div.results-main`, a vertical flex stack):

- `section.query`: a bold "Search Results" label (`.query .label`), the "Query Restated" `h1` (`.query h1`), and a paragraph of placeholder LLM output. This is where the user's restated question and the LLM summary will go.
- Three `article.exhibit-result` cards. `.exhibit-result` is a two-column grid: a `.info` panel on the left (`--gray-light` background) and a `figure` on the right.
  - The `.info` panel holds the exhibit name (`h3`), a short description taken from `documents.json`, a `.standards` paragraph (bold, small) listing "MASS STE" and the primary MA STE standard, and a `.btn` "More" link to the exhibit's mos.org page.
  - The `figure` holds the photo (`.exhibit-result img`, 16:9) and a `.credit` line. If an exhibit has no photo, the page should use `.placeholder-img` instead, which `.exhibit-result .placeholder-img` stretches to the card height.
  - The three exhibits shown are examples: Exploring AI (MS-CS-1), Engineering Design Workshop (MS-ETS1-2) and Mathematica (MS-CS-2).

Right column (`aside.results-side`, a vertical flex stack):

- `section.ask-llm.compact`: the same panel as the home page, but with the options Students, Educators (pre-selected, since this is the educator page) and Caretakers, as labelled in the wireframe.
- `section.resource-results`: `.resource-results` gives a `--gray-light` panel with a large `h2` "Resource Results". It contains six `.doc` rows. Each row is a flex row with a `.icon` (a 48 × 56 px white box with a `--blue-dark` border and the text "PDF", standing in for the document icon) and a text block with the document title (`h3`) and a one-line summary (`p`).

Images

Path	Contents
`public/images/logo.png`	The ACI Inc. logo, with a transparent background, extracted from the A1 PDF.
`public/images/exhibits/`	18 exhibit photos (1200 × 675), named after the last part of each exhibit's mos.org URL (for example `art-illusion.jpg`). They were taken from the `og:image` of each exhibit page.

The exhibit photos are Museum of Science property and are credited on the page. They are suitable for the class prototype but should be replaced or licensed before any public launch.

Not yet wired

- The Ask LLM form submits to `search-results.html`, but that page ignores the query. It still shows fixed example content. Connecting it to `ask.php` and the vector search is the next step.
- The carousel dots, calendar grid, social icons and map are visual placeholders.
- Nav links and the "Featured Exhibit" and "Learn More" buttons point to `#` anchors.
- `public/js/` is empty and is ready for the script that will call the search.

Rebuild instructions (for the Linux machine)

After `git pull`:

```
cd assignment3/webserver

# 1. Install Node dependencies (first time or after package.json changes)
npm ci

# 2. Build the vector index (needs network the first time to download the model)
npm run build-index
# → writes data/index.json (~2.4 MB)

# 3. (Optional) Re-fetch PDF extracts if source links change
python3 scripts/fetch_extracts.py
# → updates data/sources/pdf_extracts.json

# 4. (Optional) Re-fetch official site descriptions if mos.org/yes.mos.org pages change
python3 scripts/fetch_site_pages.py
# → updates data/sources/site_pages.json

# 5. (Optional) Regenerate documents.json from updated sources
python3 scripts/build_documents.py
# → updates data/documents.json
# → then re-run npm run build-index to update the vector index

# 6. (Optional) Regenerate the site navigation after editing the IA spreadsheet
python3 -m pip install openpyxl
python3 scripts/build_nav.py
# → rewrites the NAV block in public/index.html and public/search-results.html
```

`data/index.json` is gitignored. Always run `npm run build-index` after pulling.

Deployment checklist (Simmons)

```
sftp colek@web.simmons.edu
cd public_html
put ask.php ask.php
cd data
put data/index.json index.json
```

Files that must exist on the server:

Server path	Source	Notes
`public_html/ask.php`	`assignment3/webserver/public/ask.php`	The RAG endpoint
`public_html/data/index.json`	`assignment3/webserver/data/index.json`	Built locally, not in git
`/users/c/colek/.anthropic_key`	Created via SSH	`chmod 604`; never in git
`public_html/index.html`	`assignment3/webserver/public/index.html`	Main page
`public_html/search-results.html`	`assignment3/webserver/public/search-results.html`	Results page
`public_html/css/`, `public_html/images/`, `public_html/js/`	`assignment3/webserver/public/css`, `images`, `js`	Supporting files; keep the folder names so the relative links work